TEST PLAN FOR

MARKETSAFE

Note that you can refine your testing plan as the project development goes. Keep the change log as follow:

ChangeLog

Version	Change Date	By	Description
1.0	March 1st, 2026	Michael	Sprint 2 features added, Cover feature: 1,2,5
1.1	March 5th, 2026	Fidelio	added more details for the roles, more details for the CI/CD workflow description
2.0	March 23rd, 2026	Michael	Mutation Testing, Acceptance Testing
2.1	March 26th, 2026	Fidelio	Refined requirements for Sprint 3
3.0	April 1st, 2026	Michael	Load Testing

1 Introduction

1.1 Scope

Scope defines the features, functional or non-functional requirements of the software that will be tested.

The 5 features completed and tested for this sprint are **Account Management System**, **Marketplace Listings**, **Search, Offer, and Rating**. For each feature, both unit and integration testing have been implemented.

Account Management System:
The Account Management System provides secure access to the platform by only allowing users with a registered University of Manitoba email to create an account. When creating an account, the system verifies that the email is a UofM domain, is not already linked to an existing account, and that the password meets safety requirements. If these checks pass, users are given a unique user ID. Once registered and verified, users can log in and log out using JWT authentication. During login, valid credentials must generate a signed JWT containing the user ID and expiration, while invalid credentials must return proper error responses. The issued JWT must be included in any request to access protected routes. Logout must invalidate the session on the client side by removing the stored JWT, and any unauthenticated requests must be denied. Any unauthenticated user attempting to access a protected page must be redirected to the login page. A logged-in user with a valid JWT can access their own profile.

Scope tested:

- **Account Registration** — email domain validation (UofM), duplicate email check, password safety requirements, and unique user ID assignment.
- **Email Verification** — verification that the registered email is valid and belongs to a UofM domain before account activation.
- **Login and Logout** — JWT generation on valid credentials, proper error responses on invalid credentials, JWT inclusion in protected requests, client-side session invalidation on logout, and denial of requests without a valid token.
- **View Profile** — authenticated access to the user's own profile using a valid JWT, and redirection of unauthenticated users away from protected pages.

Marketplace Listings:

Marketplace Listings allows users to create, manage, and interact with listings on the platform. When creating a listing, all required information must be present before the listing is saved to the database. Once created, the listing must be visible in the marketplace. Users may delete only their own listings; deleted listings must be removed from both the database and the marketplace. Users may also comment on any posted listing, and comments from valid logged-in users must

be displayed under the respective listing. Both the commenter's own comments and other users' comments must be visible.

Scope tested:

- **Create a listing** — required field validation, successful database insertion, and visibility on the marketplace after creation.
- **Delete a listing** — permission enforcement (own listings only), removal from the database, and removal from the marketplace view.
- **Comment on a listing** — comment submission by authenticated users, display of comments under the correct listing, and visibility of both own and other users' comments.

Search:

Search enables users to find listings through keywords, and sort criteria. Users can input keywords to find matching listings, and the displayed results update whenever the keyword changes or is removed. Finally, users can sort displayed listings by a chosen criterion, and the list reorganizes accordingly when the criterion is changed.

Scope tested:

- **Search listings** — keyword matching against listings, real-time result updates on keyword change, and clearing of results when the keyword is removed.
- **Sort listings** — correct reorganization of results when a 'sort' criterion is selected, and re-sorting when the criterion is changed.

Offer:

The Offer feature allows buyers to propose a price or terms to a seller for an available listing. Upon submission, the seller receives an in-app notification indicated by a red badge on the bell icon. The seller may then accept or decline the offer. Accepting an offer changes the offer status to accepted, marks the listing as sold, and archives it from the marketplace. Declining updates the offer status to decline while leaving the listing unchanged. A buyer cannot send a new offer while one is still pending but may send another after a decline. Buyers can track the status of their offers from the Offers page.

Scope tested:

- **Send an offer** — offer creation on an available listing, proposed price or terms submission, and enforcement that a new offer cannot be sent while one is pending.
- **Offer notification** — delivery of a real-time in-app notification to the seller, displayed as a red badge on the bell icon upon receiving a new offer.
- **Accept offer** — offer status update to accepted and triggering of the mark-as-sold and archive flow.
- **Decline offer** — offer status update to decline, listing remaining unchanged, and the buyer being allowed to submit a new offer after a decline.

- **Mark listing as sold** — listing status update to sold, archival from the marketplace, and confirmed removal from available listings upon offer acceptance.

Rating:

The Rating feature allows users to share their experience with a seller by submitting a rating. Submitted ratings are saved, and the seller's average rating is calculated and displayed on their profile. Any user can view another user's rating by visiting that user's profile. Users can also view their own rating on their own profile page.

Scope tested:

- **Create a rating** — rating submission, persistent storage, and correct calculation and display of the updated average rating on the seller's profile.
- **View another user's rating** — visibility of a user's rating when visiting their profile.
- **View own rating** — visibility of the user's own rating on their personal profile page.

1.2 Roles and Responsibilities

Detail description of the Roles and responsibilities of different team members:

- **QA Analyst**
- **Test Manager & Architect**
- **Backend developer**
- **Frontend developer**
- **Backend Architect**
- **DB Designer & Architect**
- **DevOps**

Name	Net ID	GitHub username	Role
Fidelio Ciandy	Ciandyf	FidelioC	Backend Developer Frontend Developer QA Analyst DevOps – CI/CD
Farah Hegazi	Hegazif	Farahheg	Frontend Developer QA Analyst
Michael King	Kingm4	MichaelKing1619	Backend Developer Test Manager QA Analyst
Mohamed, Mohamed Youssef Mohamed	Mohammym	lMollMol	Backend Developer & Architect DB Designer & Architect Test Manager & Architect

			QA Analyst
Thomas Awad	Awadt	Tomtom4343, michel00angelo	Backend Developer QA Analyst

Role details:

- QA Analyst
    - Developed and executed unit and integration tests for both frontend and backend components.
    - Identified defects, reported issues, and ensured new features met functional and quality requirements.
- Test Manager & Architect
    - Defined the project's testing strategy, including test coverage goals and testing frameworks.
    - Coordinated testing efforts across the team and ensured CI pipelines enforced test quality standards.
- Backend developer
    - Implemented backend APIs and business logic using FastAPI.
    - Managed database interactions and ensured reliable data processing for core features such as accounts, listings, and comments.
- Frontend developer
    - Developed the user interface using Angular and integrated frontend components with backend APIs.
    - Implemented responsive layouts and user workflows for listing management and account interactions
- Backend Architect
    - Designed the backend architecture, including API structure, service layers, and system integration.
    - Ensured scalability, maintainability, and adherence to clean architecture principles..
- DB Designer & Architect
    - Designed the relational database schema for entities such as accounts, listings, and comments.
    - Defined relationships, constraints, and indexing strategies to ensure efficient and reliable data storage.
- DevOps
    - Create the CI/CD pipeline
    - Deploy application to DockerHub

2 Test Methodology

2.1 Automated regression testng

2.1.1 Test Levels

Test levels define the Types of Testing to be executed on the Application Under Test (AUT). In this course, unit testing, integraiton tesing, acceptance testing, regression testing, and laod testing are mandatory.

Requirements:

- **For unit testing, at least 10 unit tests for EACH core feature to cover the code related to each core feature**
- **For integration testing, at least 10 in total to cover core features.**
- **Acceptance testing for each core feature. Let's use end-user test for this. You can ask real end-user or your team members to go through each user story and see if the requrements are meet.**
- **For regression testing, need to execute all above unit tests + integration tests you have for each commit pushed to main branch.**

Test Level	Scope & Requirement	Methodology (How will you do this?)
Unit Testing	**[~200] tests per core feature.** Total: [~930] tests.	We use python '**unittest**' to run unit tests and use Mock and Magic Mock to mock dependencies and verfiy logic in isolation.
Integration Testing	**[129] tests total** covering interactions between features.	Using python '**unittest'** to run tests. Integration tests are ran against a test database hosted in a Docker container. In each integration test class we set up a database session and ensure tables exist and restes the tables to ensure a clean state for all test classes. MySQL DAO implementations are created allowing us to check the systems real database interactions.
Acceptance Testing	**End-user testing** for every user story.	Team members and selected end users will perform manual walkthroughs of the system based on user story acceptance criteria. For every feature, testers will follow realisic scenarios through the system UI to verify that the functionality fulfills the user requirements. During these feature walkthroughs, testers will validate expected outcomes, proper system feedback, proper authorization and general system behaviour for each task in a walkthrough.
Regression Testing	Unit + Integration tests executed on	We configured a GitHub Actions CI pipeline to automatically run our unit, integration, and frontend tests

Test Level	Scope & Requirement	Methodology (How will you do this?)
	every push to main and prod branch.	on each pull request. All CI checks must pass before a merge is allowed.

2.1.2 Unit Testing

https://github.com/MarketSafe-4350/MarketSafe/tree/main/server/tests/unit

Scope & Requirements Tested:

- **Account Management System** (Registration, email verification, login/logout, view profile) — ~172 tests, 718 statements, 100% coverage.
- **Marketplace Listings** (Create listing, delete listing, comment on listing) — ~261 tests, 874 statements, 100% coverage.
- **Search** (Search listings, filter by tags, sort listings) — embedded within Marketplace Listings.
- **Offer** (Send offer, notification, accept, decline, mark as sold) — ~192 tests, 565 statements, 100% coverage.
- **Rating** (Create rating, view others' rating, view own rating) — ~168 tests, 485 statements, 100% coverage.

Total: 930 tests, 3,486 statements, 100% coverage.

2.1.3 Integration Testing

https://github.com/MarketSafe-4350/MarketSafe/tree/main/server/tests/integration

Scope & Requirements Tested:

- **Account Management System** (Registration, email verification, login/logout, view profile) — covers database operations, account manager, account service, and the full email verification token lifecycle.
- **Marketplace Listings** (Create listing, delete listing, comment on listing) — covers the MySQL listing and comment database layers, listing and comment managers, listing service, and API route integration.
- **Search** (Search listings, filter by tags, sort listings) — embedded within Marketplace Listings integration tests.
- **Offer** (Send offer, notification, accept, decline, mark as sold) — covers the MySQL offer database layer, offer manager, offer service, and API route integration including full accept/decline/resolve flows.
- **Rating** (Create rating, view others' rating, view own rating) — covers the MySQL rating database layer and rating manager across all rating operations.

Total: 270 tests, all passed.

2.1.4 Acceptance Testing

We decided to conduct our acceptance testing manually. Each feature will have its own set of steps to follow to complete the acceptance testing. Each feature and its acceptance testing steps are as follows:

Account & Authentication:

1. Create account
    - Given I am on the registration page, when I enter newtest123@umanitoba.ca and a valid password and submit the form, then the account should be created successfully.
2. Verify email
    - Given that I have successfully registered, when I click the verification link within the valid time period, then my account should be activated.
3. Log in
    - Given I am on the login page, when I enter newtest123@umanitoba.ca and the correct password and submit, then I should be logged in and taken to the MarketSafe home page.
4. View profile
    - Given I am logged in and on the home page, when I select 'Profile', then I should see my personal account information including first and last name, email, rating (0.0), and my listings (no listings to show).
5. Log out
    - Given I am logged in and on the home page, when I select the dropdown menu in the top right and select 'Logout', then I should be redirected to the login page.

Marketplace Listings:

1. Log in
    - Given I am on the login page, when I enter test999@umanitoba.ca and password TestPass999, then I should be taken to the MarketSafe home page.
2. Comment on a listing
    - Given I am logged in and on the home page, when I select 'Test Listing', type 'Test comment!' in the comment field, and submit, then the comment should be displayed under the listing.
3. Create listing
    - Given I am logged in and on the home page, when I select '+' on My Listings, fill in the title as 'Headphones', enter a brief description, set the price to $300, set the location to University Centre, add a photo, and select 'Create', then I should see the Headphones listing appear in the marketplace with the correct price, description, and photo after refreshing.
4. Delete listing
    - Given I am logged in and viewing my Headphones listing, when I select 'Delete', then the listing should be removed and no longer visible in the marketplace after refreshing.

Search:

1. Log in
    - Given I am on the login page, when I enter test999@umanitoba.ca and password TestPass999, then I should be taken to the MarketSafe home page.
2. Search for a listing
    - Given I am logged in and on the home page, when I type 'Mountain Bike' into the search bar, then a Mountain Bike listing should be displayed as a search result.
3. Sort listings
    - Given I am logged in and on the home page, when I type '.' in the search bar and select 'Sort by Price' from the Sort By menu, then the listings should be displayed in the following order: Office Chair $150, Mountain Bike $600, iPhone 13 Pro $850, Gaming Laptop $1200.

Offer:

1. Log in
    - Given I am on the login page, when I enter test999@umanitoba.ca and password TestPass999, then I should be taken to the MarketSafe home page.
2. Send offer
    - Given I am logged in and viewing the Mountain Bike listing, when I select 'Send Offer', enter '275' in the Offered Price field and 'Drake Lobby' in the Meeting Location field, and select 'Send Offer', then the offer should be submitted successfully.
3. Offer notification
    - Given an offer has been sent to the seller, when the seller logs in, then a red notification badge should appear on the bell icon. When the seller opens the notification, then the offer details (price: $275, location: Drake Lobby) should be visible.
4. Accept offer
    - Given I am a seller viewing a received offer, when I select 'Accept', then the offer status should be updated to Accepted.
5. Mark listing as sold
    - Given I am a seller who has accepted an offer, when I or any other user browses or searches the marketplace, then the Mountain Bike listing should no longer appear in the available listings.
6. Decline offer (alternative path)
    - Given I am a seller viewing a pending offer, when I select 'Decline', then the offer status should be updated to Declined and the listing should remain visible and unchanged in the marketplace.

Rating:

1. Log in
    - Given I am on the login page, when I enter test999@umanitoba.ca and password TestPass999, then I should be taken to the MarketSafe home page.
2. Rate a seller

    - Given I am a buyer who has completed a transaction with a seller, when I submit a rating with valid details, then the rating should be saved successfully.
3. View another user's rating
    - Given a rating has been submitted, when I navigate to the seller's profile page, then the rating should be visible and accurately reflect the submitted value.
4. View own rating
    - Given I have received one or more ratings, when I navigate to my own profile page, then my rating should be displayed and reflect all ratings I have received.

2.1.5 Regression Testing (CI/CD)

Frontend CI

- Triggered automatically on pushes and pull requests to the production and main branches.
- Checks out the repository and sets up a Node.js 20 environment using GitHub Actions.
- Installs frontend dependencies using npm ci to ensure reproducible builds.
- Runs ESLint to enforce code quality and style consistency.
- Executes Angular unit tests in a headless Chrome environment.

Backend CI

- Triggered automatically on pushes and pull requests to the production and main branches.
- Sets up a Python 3.11 environment and installs project dependencies using pip.
- Performs a Python syntax compilation check (compileall) to detect early code errors.
- Executes backend **unit tests** using the Python unittest framework while collecting coverage with coverage.py.
- Runs **integration tests** and appends results to the same coverage report.
- Generates coverage reports in both terminal and HTML formats for test coverage analysis.

General Workflow

- Runs two parallel CI jobs (frontend and backend) to validate both parts of the system.
- Ensures that code changes pass linting, testing, and build verification before being merged into the main branches.

2.2 Mutation Testing (Test effectiveness)

https://github.com/MarketSafe-4350/MarketSafe/tree/main/server/tests/mutation_tests

For mutation testing we are using the 'Cosmic-Ray' framework to run our mutation tests. We chose Cosmic-Ray as it is one of the leading packages available for mutation testing python systems. Further, Cosmic-Ray integrates well with 'unittest' which is the testing framework we have chosen for our system.

To organize our testing, we separated mutation testing by feature. Each feature is associated with its own configuration file (e.g. feature_cosmic-ray.toml), which specifies the exact files to be mutated. We chose to target individual files rather than entire directories to maintain precise

control over the number of mutants generated per feature. This made it easier to track mutation coverage across features.

Mutation execution is automated using a python script that invokes Cosmic-Ray via subprocess calls for each feature's specific configuration file. Within this script, we apply several important configurations. First, we reinitialize the Cosmic-Ray session before mutating each feature to ensure that previously generated mutants are not reused or retested. Second, we use the 'cr-pragma-filter' feature which simply tells Cosmic-Ray to avoid mutating lines which contain the comment '# pragma: no mutate'. This was necessary to prevent mutation of lines such as '@override' decorators, where mutations would not be testable. After execution, Cosmic-Ray generates both .txt and HTML reports for each feature. The .txt reports allow for quick verification of mutation test result such as confirming 100% mutant kill rates, while the HTML reports provide a more detailed and visual breakdown of each mutation and its result.

Each feature individually achieved a 100% mutation score, meaning all generated mutants were killed by the existing unit tests. In total we have 405 mutants successfully killed by our tests.

2.3 Load Testing

https://github.com/MarketSafe-4350/MarketSafe/tree/main/server/tests/load_testing

For load testing, we are using the 'Locust' framework. Locust allows us to easily set up our load testing with a python script. In this python script, we created 'tasks' for the simulated users to test. Locust also opens a webpage directly from the command line which provides clear visualization of performance metrics such as request rates, response times and fail rates.

To ensure the system was fully tested, we created a task for each of our API routes, surpassing the required two per feature. This ensures that all endpoints are covered during testing. To simulate real world usage, we assigned a weight to each task based on how often that feature would typically be used. For example, searching for listings has a weight of 5 as this is very common, and commenting on a listing has a weight of 1 as this is less common.

To accurately simulate user behaviour, such as creating listings, making offers, and interacting with other users' listings, we implemented a custom setup method that runs before the load testing begins. During this setup:

- A seller account is created, logged in, and used to create a dummy listing
- A separate buyer account is created, and used by all simulated users during the tests
- The IDs of the seller account and dummy listing are stored so the simulated buyers can access the IDs and complete all tasks

This ensures that all load tests operate on the same valid account and listing and avoids creating excessive data. Creating two separate accounts ensures we can simulate user to user interactions in our load testing.

During testing, our system demonstrated strong performance with zero failures. The system handles the capacity requirement of 20 users generating 200 request per minute with no failures. Further stress testing with 200 users and roughly 2500 request per minute also showed no failures.
Here is an example of some requests used for load testing each feature:

Feature	**Request Type 1 (Read/GET)**	**Request Type 2 (Write/POST)**
Account Management System	*GET – Get my account*	*POST - Create user account*
Marketplace Listings	*GET – Get all listings*	*POST – Create new listing*
Search	*GET – Search listings*	*N/A*
Offer	*GET – Get received offers*	*POST – Resolve offer*
Rating	*GET – Get listing ratings*	*N/A*
Extra - Comments	*GET – Get listing comments*	*POST – Create listing comment*

3 Terms/Acronyms

Make a mention of any terms or acronyms used in the project

TERM/ACRONYM	DEFINITION
API	Application Program Interface
AUT	Application Under Test
DAO	Domain Access Object
UI	User Interface
QA	Quality Assurance
JWT	JSON Web Token